

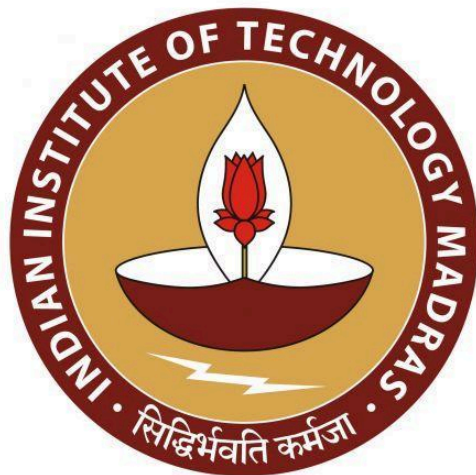
Software Engineering Project May-2026

Milestone 3 Deliverables for Campus Connect

Submitted By

Team Name: NexMind

Team Code : Team-003



IITM Online BS Degree Program,
Indian Institute of Technology, Madras, Chennai
Tamil Nadu, India, 600036

Student Club & Event Management Platform for Higher-Education Campuses

Project Details

Field	Value
Software Name	Campus Connect
Institutional Context	Campus Council / Student Club Ecosystem
System Type	Community Operations Platform (Club, Event & Participation Management)
Milestone	Milestone 3 - Sprint 1: API Endpoints, Test Cases and User Testing
Sprint Window	23 July 2026 - 2 August 2026
Date of Submission	02-08-2026
Live API Base URL	https://campusconnect.itshrestha.dev
Interactive API Documentation	https://campusconnect.itshrestha.dev/docs (Swagger UI)
Machine-readable Specification	https://campusconnect.itshrestha.dev/openapi.json
Frontend Deployment	https://campus-connect-swe.vercel.app/
Code Repository	https://github.com/Srivastava-Shrestha/MAY2026-Team-003

Team Details

Team Name: NexMind **Team Code:** Team-003

Name	Roll Number	Role in the Project
Atharv Khare	23f2004201	Lead Product Manager / Frontend Developer
Kavisha Tankle	23f1000041	Scrum Master / Product Manager
Shrestha Srivastava	23f3000168	Backend Lead / Code Reviewer
Pawan Kumar Choudhary	22f3000162	Backend Developer / Tester
Shrishti Gupta	23f2004336	Frontend Developer

Contents

1. Sprint 1 Overview and Scope
2. Milestone 3 Task Distribution
3. API Design Overview
4. List of APIs Created
5. List of APIs and Services Integrated
6. API to User-Story Traceability Matrix
7. Test Cases
8. Unit Testing with pytest
9. Defects Found - Expected vs Actual
10. Frontend and Backend Integration
11. User Feedback from the Sprint 1 Demo
12. Plan for Sprint 2
13. Submission Artefacts

1. Sprint 1 Overview and Scope

1.1 Sprint Goal

Sprint 1 turns the design frozen in Milestone 2 into a working, documented and tested REST backend, and wires the existing Vue 3 frontend to it. The sprint goal was to make the core participation spine of Campus Connect executable end to end:

- publish an event → register → check in → record a result
- and the club and membership lifecycle that feeds it: register a college → create a club → admin approval → join request → leader approval.

Every endpoint delivered in this sprint traces back to a SMART user story written in Milestone 1; nothing outside that backlog was built. The certificate engine (Stories 5.1-5.3), the AI Club Finder (Story 6.1) and the activity leaderboard (Story 7.1) remain deliberately out of Sprint 1 scope and are scheduled for Sprint 2, as planned in the Milestone 2 sprint schedule.

1.2 Sprint Window and Cadence

Item	Detail
Sprint 1 window	23 July 2026 - 2 August 2026 (11 days)
Target release	Milestone 3 - 2 August 2026
Stand-up	Daily asynchronous status collection at 9:00 AM and 9:00 PM on the team channel; a live stand-up on Google Meet whenever a blocker is raised
Sprint planning	22 July 2026 - backlog walk-through and ownership confirmation
Sprint review / client demo	3 August 2026 - live walkthrough with primary users
Sprint retrospective	4 August - 5 August 2026
Tracking board	Trello - https://trello.com/b/M4qafINT/nexmind

1.3 Stories Committed in Sprint 1

Story	Title	Module	Endpoints	Status
1.1	Register a college on the platform	Identity & Access	1	Delivered
1.2	Sign up and log in	Identity & Access	2	Delivered
2.1	Browse the club directory	Club Management	1	Delivered
2.2	View a club profile	Club Management	1	Delivered
2.3	Request to join a club	Membership	2	Delivered
2.4	Manage membership join requests	Membership	3	Delivered
2.5	Create a club and request approval	Club Management	3	Delivered
2.6	Approve or reject club registration	Club Management	2	Delivered
3.1	Create and publish an event	Event Management	3	Delivered
3.2	Register for an event	Event Management	4	Delivered
3.3	View registered participants	Event Management	1	Delivered
3.4	Mark attendance at an event	Event Management	1	Delivered
3.5	Set a result for each attendee	Event Management	1	Delivered
3.6	View my event result	Event Management	2	Delivered

Story	Title	Module	Endpoints	Status
3.7	Edit or cancel an event	Event Management	2	Delivered
4.1	Post a club announcement	Communication	4	Delivered
4.2	View announcements from joined clubs	Communication	3	Delivered
4.3	Raise an issue or query	Communication	2	Delivered
4.4	View and resolve member issues	Communication	4	Delivered
6.1	Find clubs by describing my interests (AI Club Finder)	AI Discovery	-	[[STATUS: STORY_6_1]] - carried to Sprint 2
-	Notification inbox supporting stories 2.4, 3.2 and 3.5	Communication	4	Delivered

Story 6.1 (AI Club Finder) was committed to Sprint 1 in the Milestone 2 plan. The deterministic scoring core and its 23-case pytest suite are complete; the thin backend proxy that holds the Anthropic API key is not yet merged, so the story is reported honestly as carried over rather than shown as delivered.

1.4 Sprint 1 Outcome at a Glance

Metric	Value
User stories committed	20 (19 core + AI Club Finder)
User stories delivered	19
API operations created by the team	45
Distinct resource paths	37
Functional modules covered	6 of 8 (Identity & Access, Club Management, Membership, Event Management, Communication, Notifications)
Test cases designed	399
pytest cases executed	399
Defects found where expected $\neq$ actual	4 functional + 1 documentation
Defects fixed and re-verified	10

1.5 Milestone 3 Rubric Coverage

Where each item required by the course rubric for Sprint 1 is answered in this report:

Rubric requirement	Section	Owner
List of APIs created	§4	Shrestha
List of APIs integrated	§5	Shrishti
Description of API endpoints	§3, §4	Shrestha
Swagger-compatible YAML documentation	§3.6 + campus-connect-openapi.yaml	Shrestha, Kavisha
Backend code of the implemented APIs	Submitted zip - backend/	Shrestha, Pawan
Mapping of APIs to user stories	§6	Shrestha
Extensive test cases (input / expected / actual / result)	§7	Pawan, Kavisha

Rubric requirement	Section	Owner
Case where expected $\neq$ actual, with the fix	§9	Pawan, Kavisha
pytest code and output	§8 + backend/tests/	Pawan
User feedback on the delivered features	§11	Atharv, Kavisha
Plan for the next sprint	§12	Atharv, Kavisha
Facilitating and composing Milestone 3 submission deliverables	-	Atharv, Kavisha

2. Milestone 3 Task Distribution

Roles were assigned in Milestone 2 and carried unchanged into Sprint 1. The table below records what each member owned during this sprint and which artefact in this submission they are accountable for.

Member	Role	Sprint 1 work owned	Milestone 3 artefact owned
Atharv Khare	Lead Product Manager / Frontend Developer	Sprint planning and daily coordination; story-to-endpoint traceability; user acceptance testing of every delivered story; running the Sprint 1 demo with primary users; scope control on Story 6.1	This report (assembly and final review), §1, §6, §11, §12; combined-zip validity check
Kavisha Tankle	Scrum Master / QA Engineer	Board hygiene on Trello; blocker resolution; authored the endpoint test matrix covering happy path, authorisation and validation; manual lifecycle validation of the participation spine against Swagger	§7 test-case matrix, §9 defect log, §13 scrum minutes, GitHub issue records
Shrestha Srivastava	Backend Lead / Code Reviewer	Database schema on Neon Postgres; JWT authentication and role guards; Identity & Access, Club Management and Membership endpoints; review and merge of every pull request	§3 API design, §4 endpoint catalogue, campus-connect-openapi.yaml, backend/ source
Pawan Kumar Choudhary	Backend Developer / Tester	Event lifecycle, announcement, issue-tracking and notification endpoints; pytest fixtures and the automated endpoint suite; fix verification for the defects in §9	§5 integrated services, §8 pytest suite and screenshots, backend/tests/
Shrishti Gupta	Frontend Developer	Replacing frontend mocks with live API calls; auth token handling and route guards; role-based dashboards wired to the new endpoints; error and empty-state handling	§10 integration screenshots and the integrated frontend/ build

3. API Design Overview

3.1 Environments

Environment	Base URL	Notes
Production (Render.com)	https://campusconnect.itshrestha.dev	FastAPI service backed by Neon serverless PostgreSQL
Interactive docs	https://campusconnect.itshrestha.dev/docs	Swagger UI, generated from the same spec that is submitted as YAML
OpenAPI specification	https://campusconnect.itshrestha.dev/openapi.json	OpenAPI 3.1.0, exported to YAML for submission
Local development	http://127.0.0.1:8000	uvicorn app.main:app --reload

3.2 Authentication and Authorisation

Authentication is stateless. A successful signup or login returns a short-lived JWT access token and a refresh token; every protected route expects the access token in an Authorization: Bearer header. Tokens are signed with PyJWT and carry the user id, the college id and the platform role.

Authorisation is enforced at two distinct levels, and the distinction matters when reading the access column in §4:

- **Platform role** - STUDENT or CAMPUS_ADMIN. This is a property of the user account and is checked by a FastAPI dependency before the handler runs. Only CAMPUS_ADMIN may onboard a college or approve/reject a club.
- **Membership role** - LEADER or MEMBER. This is a property of a membership row, not of the account, so a single student can lead one club and simply be a member of another. Leader-only actions (publishing events, marking attendance, replying to issues) are therefore declared as STUDENT in the specification and are gated inside the handler against the membership record for that specific club.
- **College scoping** - every query is filtered by the college id carried in the token, so a student of one institution can never read or act on another institution's clubs, events or members. A cross-college read is answered with 404 rather than 403 so the existence of another college's records is not leaked.

3.3 Request and Response Conventions

- JSON in, JSON out. All timestamps are ISO-8601 in UTC.
- Collection endpoints accept limit and offset for pagination and return a flat array; list items are trimmed projections while the detail endpoint returns the full record.
- Write endpoints return a small acknowledgement object carrying the affected id, the resulting status and a human-readable message, so the frontend can update state without a second round trip.
- State transitions are exposed as explicit verbs (PATCH .../publish, .../cancel, .../approve, .../resolve) rather than a generic status field, which keeps every entity on the single-entity-plus-status-flow pattern the whole system is built on.
- Enumerations are transported as uppercase strings and validated by Pydantic, so an invalid value fails at the boundary with a 422 rather than reaching the database.

3.4 Error Handling

Status	Meaning in Campus Connect	Representative case
200 OK	Request succeeded; response body carries the resource or acknowledgement	GET /clubs, POST /events/{id}/register

Status	Meaning in Campus Connect	Representative case
400 Bad Request	Semantically invalid request that passed schema validation	Signup on an email domain no college has registered
401 Unauthorized	Missing, malformed or expired bearer token; wrong credentials	POST /auth/login with a wrong password
403 Forbidden	Authenticated but not permitted - wrong platform role or not the club leader	A member trying to publish an event
404 Not Found	Resource does not exist, or exists outside the caller's college	GET /clubs/{id} for another college's club
409 Conflict	Request contradicts the current state of the resource	Registering for a full event; approving an already-approved club
422 Unprocessable Entity	Schema or field-constraint violation raised by Pydantic	capacity = 0, ends_at earlier than starts_at

Known gap to close before Milestone 4: the generated specification currently declares only 200 and 422 responses for each operation because the non-2xx paths are raised as HTTPException without a declared responses= mapping on the route decorator. The behaviour is correct at runtime and is evidenced in §7, but the YAML must document it. This is logged as DOC-01 in §9 and owned by Shrestha.

3.5 Domain Enumerations

Enumeration	Values	Used by
UserRole	STUDENT, CAMPUS_ADMIN	Signup, all role guards
ClubType	OFFICIAL, UNOFFICIAL	Club creation and directory filters
ClubStatus	PENDING, ACTIVE, REJECTED, ARCHIVED	Club approval workflow
MembershipRole	LEADER, MEMBER	Membership records, leader-only guards
MembershipStatus	PENDING, APPROVED, REJECTED	Join-request workflow
EventStatus	DRAFT, PUBLISHED, CANCELLED	Event lifecycle
RegistrationResult	REGISTRANT, PARTICIPANT, RUNNER_UP, WINNER	Result recording, certificate eligibility in Sprint 2
AnnouncementCategory	GENERAL, EVENT_UPDATE, RESOURCE, ACHIEVEMENT, URGENT	Announcement feed
IssueCategory	EVENT, CLUB, CERTIFICATE, TECHNICAL, GENERAL	Issue tracking
IssueStatus	OPEN, IN_PROGRESS, RESOLVED	Issue workflow
NotificationType	JOIN_APPROVED, JOIN_REJECTED, REGISTRATION_CONFIRMED, RESULT_POSTED, CERTIFICATE_ISSUED	Notification inbox

3.6 Swagger-Compatible YAML Documentation

The specification is generated by FastAPI from the Pydantic request and response models, so it can never drift from the implementation. It is exported to YAML for submission with:

```
curl -s https://campusconnect.itshrestha.dev/openapi.json | python -c "import sys,json,yaml;yaml.safe_dump(json.load(sys.stdin),sys.stdout,sort_keys=False)" > campus-connect-openapi.yaml
```

- File submitted as campus-connect-openapi.yaml (OpenAPI 3.1.0, Swagger-compatible).
- It documents all 45 operations across 37 paths, the HTTPBearer security scheme, every request and response schema, and the per-operation role scopes.
- The same document renders as the live Swagger UI at /docs, which was used by the QA lead for manual lifecycle validation.

[[IMAGE: SWAGGER_UI]] [\[Link\]](#)

Screenshot of the live Swagger UI at <https://campusconnect.itshrestha.dev/docs> showing the seven tag groups expanded

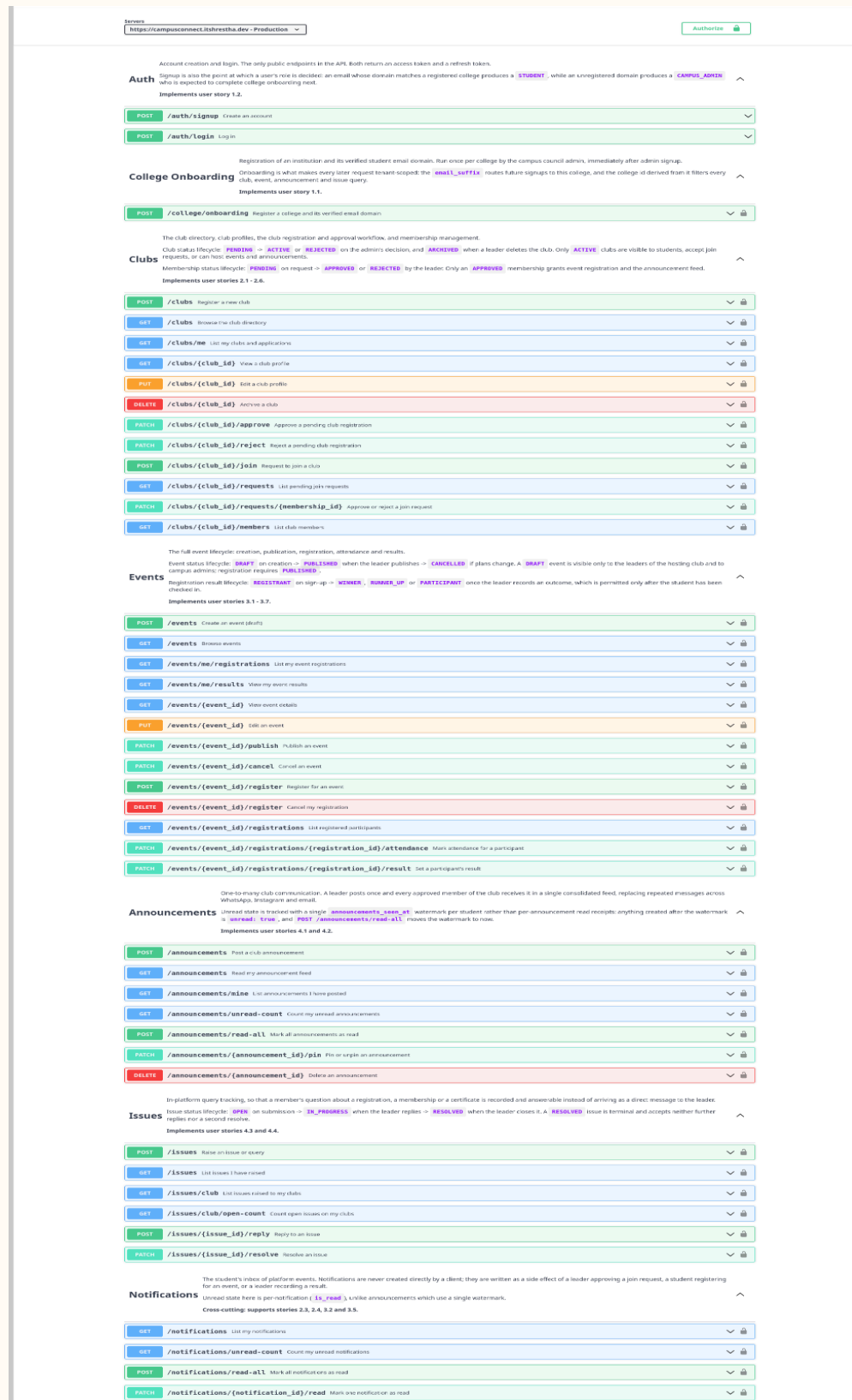


Figure 1: Swagger UI generated from the submitted specification

4. List of APIs Created

All 45 operations below were designed and implemented by the team in Sprint 1. The Access column gives the platform role required by the route guard; where a route is marked STUDENT but the action is a leader action, the membership-level check described in §3.2 applies inside the handler.

4.1 Auth (2 operations)

#	Method	Endpoint	Description	Access	Story
1	POST	/auth/signup	Creates a student or campus-admin account. Email domain is matched against a registered college; returns a JWT access token and refresh token on success.	Public	1.2
2	POST	/auth/login	Authenticates an existing user against hashed credentials and issues a fresh access/refresh token pair.	Public	1.2

4.2 College Onboarding (1 operations)

#	Method	Endpoint	Description	Access	Story
3	POST	/college/onboarding	Registers an institution and locks its official student email suffix, so later signups on that domain are auto-verified into the college space.	CAMPUS_ADMIN	1.1

4.3 Clubs (12 operations)

#	Method	Endpoint	Description	Access	Story
4	POST	/clubs	Submits a new club registration. The club is created in PENDING state and enters the campus-admin approval queue; the creator becomes its LEADER.	STUDENT	2.5
5	GET	/clubs	Returns the club directory for the caller's college with optional status / type / category filters and free-text search.	STUDENT / CAMPUS_ADMIN	2.1
6	GET	/clubs/me	Returns the clubs the caller belongs to, with membership role and membership status; filterable by role or status.	STUDENT	2.3, 2.5
7	GET	/clubs/{club_id}	Returns a full club profile - description, category, type, member count, links and club head details.	STUDENT / CAMPUS_ADMIN	2.2
8	PUT	/clubs/{club_id}	Updates the editable fields of a club (description, category, image, links). Restricted to the club LEADER.	STUDENT	2.5

#	Method	Endpoint	Description	Access	Story
9	DELETE	/clubs/{club_id}	Archives a club. Restricted to the club LEADER; membership and event history is retained for audit.	STUDENT	2.5
10	PATCH	/clubs/{club_id}/approve	Campus-admin approval of a pending club registration; moves the club to ACTIVE and makes it visible in the directory.	CAMPUS_ADMIN	2.6
11	PATCH	/clubs/{club_id}/reject	Campus-admin rejection of a pending club registration; moves the club to REJECTED and notifies the applicant.	CAMPUS_ADMIN	2.6
12	POST	/clubs/{club_id}/join	Raises a membership join request for an ACTIVE club. Membership is created in PENDING state for leader review.	STUDENT	2.3
13	GET	/clubs/{club_id}/requests	Returns the pending join-request queue for a club. Restricted to the club LEADER.	STUDENT	2.4
14	PATCH	/clubs/{club_id}/requests/{membership_id}	Approves or rejects a single join request and raises a JOIN_APPROVED / JOIN_REJECTED notification for the applicant.	STUDENT	2.4
15	GET	/clubs/{club_id}/members	Returns the approved member roster of a club with each member's membership role.	STUDENT	2.4

4.4 Events (13 operations)

#	Method	Endpoint	Description	Access	Story
16	POST	/events	Creates an event in DRAFT state against a club (title, description, venue, start/end time, capacity). Restricted to the club LEADER.	STUDENT	3.1
17	GET	/events	Returns the event feed for the caller's college with status, club, search and upcoming-only filters; includes seats-left computation.	STUDENT / CAMPUS_ADMIN	3.1, 3.2
18	GET	/events/me/registrations	Returns every event the caller has registered for, with check-in state and recorded result.	STUDENT	3.2, 3.6
19	GET	/events/me/results	Returns the caller's participation record - the result (Winner / Runner-up / Participant) for every event attended.	STUDENT	3.6
20	GET	/events/{event_id}	Returns a full event detail view including registration count, seats left and whether the caller is already registered.	STUDENT / CAMPUS_ADMIN	3.2
21	PUT	/events/{event_id}	Updates event details. Restricted to the club LEADER; registered students are notified of the change.	STUDENT	3.7
22	PATCH	/events/{event_id}/publish	Moves an event from DRAFT to PUBLISHED, making it registrable and pushing it to club members' feeds.	STUDENT	3.1

#	Method	Endpoint	Description	Access	Story
23	PATCH	/events/{event_id}/cancel	Cancels a published event, closes registration and notifies every registered student.	STUDENT	3.7
24	POST	/events/{event_id}/register	Registers the caller for a published event and returns a confirmation carrying a unique registration ID. Enforces the capacity limit.	STUDENT	3.2
25	DELETE	/events/{event_id}/register	Withdraws the caller's registration and releases the seat back to the capacity pool.	STUDENT	3.2
26	GET	/events/{event_id}/registrations	Returns the full participant list for an event with roll number, branch, year, check-in state and result. Restricted to the club LEADER.	STUDENT	3.3
27	PATCH	/events/{event_id}/registrations/{registration_id}/attendance	Marks a registered student as checked in (or reverses it) and stamps the check-in time. Restricted to the club LEADER.	STUDENT	3.4
28	PATCH	/events/{event_id}/registrations/{registration_id}/result	Assigns WINNER / RUNNER_UP / PARTICIPANT to a checked-in attendee and raises a RESULT_POSTED notification.	STUDENT	3.5

4.5 Announcements (7 operations)

#	Method	Endpoint	Description	Access	Story
29	POST	/announcements	Publishes a categorised announcement to all members of a club, optionally pinned. Restricted to the club LEADER.	STUDENT	4.1
30	GET	/announcements	Returns the consolidated announcement feed across every club the caller has joined, pinned items first, with pagination.	STUDENT	4.2
31	GET	/announcements/mine	Returns announcements authored by the caller's own club(s), used by the leader dashboard.	STUDENT	4.1
32	GET	/announcements/unread-count	Returns the number of announcements published since the caller last marked the feed as read - drives the navbar badge.	STUDENT	4.2
33	POST	/announcements/read-all	Stamps the caller's announcement seen-at marker, clearing the unread badge.	STUDENT	4.2
34	PATCH	/announcements/{announcement_id}/pin	Pins or unpins an announcement to the top of the club feed. Restricted to the club LEADER.	STUDENT	4.1
35	DELETE	/announcements/{announcement_id}	Deletes an announcement. Restricted to the leader of the club that authored it.	STUDENT	4.1

4.6 Issues (6 operations)

#	Method	Endpoint	Description	Access	Story
36	POST	/issues	Raises a tracked issue or query against a club, optionally linked to a specific event, in one of five categories.	STUDENT	4.3
37	GET	/issues	Returns the caller's own raised issues with the leader's response and resolution timestamp.	STUDENT	4.3
38	GET	/issues/club	Returns the incoming issue queue for the clubs the caller leads, filterable by status and club.	STUDENT	4.4
39	GET	/issues/club/open-count	Returns the count of unresolved issues across the caller's clubs - drives the leader dashboard badge.	STUDENT	4.4
40	POST	/issues/{issue_id}/reply	Records a leader's written response on an issue and moves it to IN_PROGRESS.	STUDENT	4.4
41	PATCH	/issues/{issue_id}/resolve	Closes an issue, stamps the resolution time and notifies the student who raised it.	STUDENT	4.4

4.7 Notifications (4 operations)

#	Method	Endpoint	Description	Access	Story
42	GET	/notifications	Returns the caller's notification inbox (join outcomes, registration confirmations, results), newest first, with read/type filters.	STUDENT	2.4, 3.2, 3.5
43	GET	/notifications/unread-count	Returns the unread notification count for the bell badge.	STUDENT	2.4, 3.2, 3.5
44	POST	/notifications/read-all	Marks every notification for the caller as read and returns the number updated.	STUDENT	2.4, 3.2, 3.5
45	PATCH	/notifications/{notification_id}/read	Marks a single notification as read.	STUDENT	2.4, 3.2, 3.5

5. List of APIs and Services Integrated

In addition to the endpoints created by the team, Campus Connect consumes the third-party services below. Nothing in this list replaces team-written business logic; each is used for a single, well-defined capability.

Service / API	Purpose	Where it is used	Status in Sprint 1
Neon serverless PostgreSQL	Managed relational store for every entity in the ER design	All backend modules, via SQLAlchemy	Live
PyJWT	Signing and verification of access and refresh tokens	POST /auth/signup, POST /auth/login and every protected route	Live
Resend Email API	Transactional email - signup confirmation and event reminders	Identity & Access and Event Management notification fan-out	pending
Anthropic Messages API (Claude Haiku)	Free-text reason strings for the AI Club Finder; deterministic scoring stays in our own code	Story 6.1 - thin backend proxy so the API key is never exposed to the browser	Carried to Sprint 2
cairosvg + Jinja2 + qrcode[pil]	SVG-to-PDF certificate rendering with an embedded verification QR code	Stories 5.1-5.3, Certificate module	Scheduled for Sprint 2
Render.com	Backend hosting and automatic deploys from the main branch	Deployment pipeline	Live
Vercel	Frontend hosting and preview deployments per branch	Deployment pipeline	Live

The AI Club Finder is the only GenAI integration in the product. Its architecture is already frozen: a deterministic scoring function ranks clubs against a student's stated interests, and Claude Haiku is used only to write the short human-readable reason attached to each recommendation. The model never decides the ranking, and the key is held server-side behind our own proxy endpoint.

6. API to User-Story Traceability Matrix

Every endpoint in §4 exists because a Milestone 1 user story required it. The matrix below is read in the other direction - story first - so that no committed story is left without an implementation and no endpoint exists without a story behind it.

Story	User story title	Endpoints implementing it	Owner	Status	Test cases
1.1	Register a college on the platform	POST /college/onboarding	Shrestha	Delivered	IT-172 – IT-193
1.2	Sign up and log in	POST /auth/signup; POST /auth/login	Shrestha	Delivered	IT-057 – IT-083
2.1	Browse the club directory	GET /clubs	Shrestha	Delivered	IT-108 – IT-118
2.2	View a club profile	GET /clubs/{club_id}	Shrestha	Delivered	IT-119 – IT-125
2.3	Request to join a club	POST /clubs/{club_id}/join; GET /clubs/me	Shrestha	Delivered	IT-330 – IT-336, IT-160 – IT-171
2.4	Manage membership join requests	GET /clubs/{club_id}/requests; PATCH /clubs/{club_id}/requests/{membership_id}; GET /clubs/{club_id}/members	Shrestha	Delivered	IT-337 – IT-354
2.5	Create a club and request approval	POST /clubs; PUT /clubs/{club_id}; DELETE /clubs/{club_id}	Shrestha	Delivered	IT-084 – IT-107, IT-126 – IT-148
2.6	Approve or reject club registration	PATCH /clubs/{club_id}/approve; PATCH /clubs/{club_id}/reject	Shrestha	Delivered	IT-149 – IT-159
3.1	Create and publish an event	POST /events; PATCH /events/{event_id}/publish; GET /events	Shrestha	Delivered	IT-194 – IT-202, IT-203 – IT-207, IT-220 – IT-228
3.2	Register for an event with confirmation	POST /events/{event_id}/register; DELETE /events/{event_id}/register; GET /events/{event_id}; GET /events/me/registrations	Shrestha	Delivered	IT-233 – IT-245, IT-258 – IT-260
3.3	View registered participants	GET /events/{event_id}/registrations	Shrestha	Delivered	IT-246 – IT-247
3.4	Mark attendance at an event	PATCH /events/{event_id}/registrations/{registration_id}/attendance	Shrestha	Delivered	IT-248 – IT-252
3.5	Set a result for each attendee	PATCH /events/{event_id}/registrations/{registration_id}/result	Shrestha	Delivered	IT-253 – IT-257
3.6	View my event result	GET /events/me/results; GET /events/me/registrations	Shrestha	Delivered	IT-261 – IT-263
3.7	Edit or cancel an event	PUT /events/{event_id}; PATCH /events/{event_id}/cancel	Shrestha	Delivered	IT-208 – IT-219
4.1	Post a club announcement	POST /announcements; GET /announcements/mine; PATCH /announcements/{id}/pin; DELETE /announcements/{id}	Shrestha	Delivered	IT-001 – IT-016, IT-028 – IT-030, IT-035 – IT-044, IT-056
4.2	View announcements from joined clubs	GET /announcements; GET /announcements/unread-count; POST /announcements/read-all	Shrestha	Delivered	IT-017 – IT-027, IT-031 – IT-034, IT-045 – IT-055
4.3	Raise an issue or query	POST /issues; GET /issues	Shrestha	Delivered	IT-264 – IT-291

Story	User story title	Endpoints implementing it	Owner	Status	Test cases
4.4	View and resolve member issues	GET /issues/club; GET /issues/club/open-count; POST /issues/{id}/reply; PATCH /issues/{id}/resolve	Shrestha	Delivered	IT-292 – IT-329
2.4, 3.2, 3.5	Notification inbox supporting the above stories	GET /notifications; GET /notifications/unread-count; POST /notifications/read-all; PATCH /notifications/{id}/read	Shrestha	Delivered	IT-355 – IT-392
5.1 - 5.3	Certificate generation, download and verification	Not yet implemented	Shrestha	Sprint 2	-
6.1	AI Club Finder	Backend proxy pending merge	Atharv, Pawan	Sprint 2	-
7.1	Club activity leaderboard	Not yet implemented	Pawan	Sprint 2	-

7. Test Cases [\[Link of the full test cases document\]](#)

Test cases are written in the format required by the course - API being tested, inputs, expected output, actual output and result. 399 cases were designed across the six modules delivered in Sprint 1, covering the happy path, authentication, authorisation, tenant isolation, input validation and state-transition conflicts for every endpoint. Please find all the 399 test cases in the document whose link is attached in the heading of this section.

The Actual Output column records the observed behaviour of the deployed build at the end of Sprint 1. Rows that reference a defect id (BUG-01 to BUG-04) failed on the first run and passed only after the fix described in §9.

7.1 Identity & Access (Stories 1.1, 1.2) - 13 cases

ID	API	Inputs	Expected Output	Actual Output	Result
IT-057	POST /auth/signup	<code>{"email": "pawan.kumar@knit.edu.in", "full_name": "Pawan Kumar", "password": "Pawan@123", "confirm_password": "Pawan@123", "role": "STUDENT"}</code>	200; msg 'Account created successfully'; access_token, refresh_token	Status 200; msg 'Account created successfully'; tokens present	Success
IT-058	POST /auth/signup	<code>{"email": "admin@newcollege.edu", "full_name": "Admin User", "password": "Admin@123", "confirm_password": "Admin@123", "role": "CAMPUS_ADMIN"}</code>	200; msg 'Account created successfully'; tokens	200; msg 'Account created successfully'; tokens	Success
IT-059	POST /auth/signup	<code>{"email": "student@cs.knit.edu.in", "full_name": "Subdomain Student", "password": "Test@1234", "confirm_password": "Test@1234", "role": "STUDENT"}</code>	200; msg 'Account created successfully'; tokens	200; msg 'Account created successfully'; tokens	Success
IT-060	POST /auth/login	<code>{"email": "pawan.kumar@knit.edu.in", "password": "Pawan@123"}</code> (login with lowercase email after signing up with uppercase email)	200; msg 'Logged in successfully'; tokens	Status: 401; msg 'Incorrect email or password'	Fail
IT-061	POST /auth/signup	full_name exactly 100 chars (see test code)	200; msg 'Account created successfully'; tokens	200; msg 'Account created successfully'; tokens	Success
IT-062	POST /auth/signup	<code>{"email": "someone@unknown-domain.edu", "full_name": "Someone Unknown", "password": "Test@1234", "confirm_password": "Test@1234", "role": "STUDENT"}</code>	404; msg 'College not registered'	404; msg 'College not registered'	Success

7.2 Club Management & Membership (Stories 2.1 - 2.6) - 28 cases

ID	API	Inputs	Expected Output	Actual Output	Result
IT-084	POST /clubs	<code>{"name": "Robotics Club", "description": "A club for robotics enthusiasts and builders", "category": "Technical", "type": "OFFICIAL"}</code>	200; status PENDING; msg 'Club submitted for admin approval'	200; status PENDING; msg 'Club submitted for admin approval'	Success
IT-085	POST /clubs	<code>{"name": "Chess Circle", "description": "A club for chess enthusiasts and casual players", "category": "Games", "type": "UNOFFICIAL"}</code>	200; status ACTIVE; msg 'Club created and is now live'	200; status ACTIVE; msg 'Club created and is now live'	Success
IT-086	POST /clubs	<code>{"name": "Linkless Club", "description": "A club created without any links", "category": "Technical", "type": "UNOFFICIAL", "links": []}</code>	200	200	Success
IT-087	POST /clubs	<code>{"name": "Abc", "description": "A club with a very short name for testing", "category": "Technical", "type": "UNOFFICIAL"}</code>	200; name=='Abc'	200; name=='Abc'	Success
IT-088	POST /clubs	<code>{"name": "Ab", "description": "A club with a name that is too short", "category": "Technical", "type": "UNOFFICIAL"}</code>	422	422	Success
IT-089	POST /clubs	<code>{'name': 'A'*100, 'description': 'A club with a name exactly at the max length', 'category': 'Technical', 'type': 'UNOFFICIAL'}</code>	200; name=='A'*100	200; name=='A'*100	Success
IT-090	POST /clubs	<code>{'name': 'A'*101, 'description': 'A club with a name exceeding the max length', 'category': 'Technical', 'type': 'UNOFFICIAL'}</code>	422	422	Success
IT-091	POST /clubs	<code>{"name": "Music Society", "description": "Hello", "category": "Arts", "type": "UNOFFICIAL"}</code>	200	200	Success

7.3 Event Lifecycle (Stories 3.1 - 3.7) - 28 cases

ID	API	Inputs	Expected Output	Actual Output	Result
		'starts_at': future_time(48), 'ends_at': future_time(50), 'capacity': 30}			
IT-195	POST /events	{'club_id': club_id, 'title': 'Open Mic Night', 'description': 'An open evening for anyone who wants to perform', 'venue': 'Amphitheatre', 'starts_at': future_time(72), 'ends_at': future_time(75)}	200; status DRAFT	200; status DRAFT	Success
IT-196	POST /events	{'club_id': club_id, 'title': 'Time Travel Session', ..., 'starts_at': future_time(50), 'ends_at': future_time(48)}	422	422	Success
IT-197	POST /events	{'club_id': club_id, 'title': 'Zero Seat Event', ..., 'capacity': 0}	422	422	Success
IT-198	POST /events	{'club_id': club_id, 'title': 'AB', ...}	422	422	Success
IT-199	POST /events	{'club_id': club_id}	422	422	Success
IT-200	POST /events	{'club_id': 999999, 'title': 'Ghost Club Event', ...}	404; msg 'Club not found'	404; msg 'Club not found'	Success
IT-201	POST /events	{'club_id': club_id, 'title': 'Unauthorised Event', ...} (member)	403	403	Success
IT-202	POST /events	{'club_id': club_id, 'title': 'Anonymous Event', ...} (no token)	401	401	Success
IT-203	PATCH /events/{id}/publish	leader publishing their draft event	200; status PUBLISHED; msg 'Event published successfully'	200; status PUBLISHED; msg 'Event published successfully'	Success
IT-204	PATCH /events/{id}/publish	N/A (start already past)	403; msg 'An event starting in the past cannot be published'	403; msg 'An event starting in the past cannot be published'	Success

7.4 Communication, Issues & Notifications (Stories 4.1 - 4.4) - 25 cases

ID	API	Inputs	Expected Output	Actual Output	Result
IT-001	POST /announcements	{'club_id': club_id, 'title': 'Lab equipment rules', 'body': 'Book slots in the register before use', 'category': 'RESOURCE'}	200; club_id, title, category match; is_pinned False; msg 'Announcement posted successfully'	200; club_id, title, category match; is_pinned False; msg 'Announcement posted successfully'	Success
IT-002	POST /announcements	{'club_id': club_id, 'title': 'Urgent maintenance notice', 'body': 'The lab will be closed this weekend for maintenance', 'category': 'URGENT', 'is_pinned': True}	200; is_pinned True	200; is_pinned True	Success
IT-003	POST /announcements	{'club_id': club_id, 'title': 'Abc', 'body': 'A short title at the minimum boundary', 'category': 'GENERAL'}	200; title=='Abc'	200; title=='Abc'	Success
IT-004	POST /announcements	{'club_id': club_id, 'title': 'Ab', 'body': 'A title that is one character too short', 'category': 'GENERAL'}	422	422	Success
IT-005	POST /announcements	{'club_id': club_id, 'title': 'A'*150, 'body': 'A title exactly at the maximum length boundary', 'category': 'GENERAL'}	200; title=='A'*150	200; title=='A'*150	Success
IT-006	POST /announcements	{'club_id': club_id, 'title': 'A'*151, 'body': 'A title exceeding the maximum length boundary', 'category': 'GENERAL'}	422	422	Success
IT-007	POST /announcements	{'club_id': club_id, 'title': 'Short body test', 'body': 'Hello', 'category': 'GENERAL'}	200	200	Success
IT-008	POST /announcements	{'club_id': club_id, 'title': 'Too short body test', 'body': 'Hi', 'category': 'GENERAL'}	422	422	Success
IT-009	POST /announcements	{'club_id': club_id, 'title': 'Too long body test', 'body': 'A'*5001, 'category': 'GENERAL'}	422	422	Success

7.5 Snap shots of Pytest functions

```
@pytest.mark.asyncio
async def test_feed_returns_approved_member_announcements(client, leader, member):
    """Verify that an approved member sees announcements from their joined club in the feed"""
    headers, club_id = leader
    payload = {
        "club_id": club_id,
        "title": "Feed visibility test",
        "body": "This should show up in the member's feed",
        "category": "GENERAL",
    }
    posted = await client.post("/announcements", headers=headers, json=payload)
    announcement_id = posted.json()["id"]

    response = await client.get("/announcements", headers=member)
    assert response.status_code == 200
    body = response.json()
    ids = [item["id"] for item in body]
    assert announcement_id in ids
    item = next(i for i in body if i["id"] == announcement_id)
    assert item["club_id"] == club_id
    assert item["club_name"] == "Robotics Club"
    assert item["author_name"] == "Club Leader"
    assert item["unread"] is True
```

```

@pytest.mark.asyncio
async def test_signup_max_length_name_succeeds(client, seed_college):
    """Validate that signup succeeds with a full name exactly at the 100-character limit"""
    payload = {
        "email": "longname@knit.edu.in",
        "full_name": "A" * 100,
        "password": "Test@1234",
        "confirm_password": "Test@1234",
        "role": "STUDENT"
    }
    response = await client.post("/auth/signup", json=payload)
    assert response.status_code == 200
    body = response.json()
    assert "access_token" in body
    assert isinstance(body["access_token"], str)
    assert len(body["access_token"]) > 0
    assert "refresh_token" in body
    assert isinstance(body["refresh_token"], str)
    assert len(body["refresh_token"]) > 0
    assert body["message"] == "Account created successfully"

```

```

@pytest.mark.asyncio
async def test_browse_events_hides_drafts_from_students(client, leader, outsider):
    """Ensure that a draft event is excluded from a student's browse results"""
    leader_headers, club_id = leader
    payload = {
        "club_id": club_id,
        "title": "Unpublished Workshop",
        "description": "This event has never been published",
        "venue": "Lab 204, Main Block",
        "starts_at": future_time(48),
        "ends_at": future_time(50),
    }
    create = await client.post("/events", headers=leader_headers, json=payload)
    event_id = create.json()["id"]

    response = await client.get("/events", headers=outsider)
    assert response.status_code == 200
    ids = [event["id"] for event in response.json()]
    assert event_id not in ids

```

```

@pytest.mark.asyncio
async def test_browse_clubs_search_is_case_insensitive(client, student_token):
    """Verify that the search filter matches club names regardless of casing"""
    payload = {
        "name": "Photography Guild",
        "description": "A club for photography enthusiasts",
        "category": "Arts",
        "type": "UNOFFICIAL"
    }
    await client.post("/clubs", json=payload, headers={"Authorization": f"Bearer {student_token}"})

    response = await client.get(
        "/clubs", params={"search": "PHOTOGRAPHY"}, headers={"Authorization": f"Bearer {student_token}"})
    assert response.status_code == 200
    names = [club["name"] for club in response.json()]
    assert "Photography Guild" in names

```

```
@pytest.mark.asyncio
async def test_onboarding_short_name_fails(client, admin_token):
    """Validate that college onboarding is rejected when the college name is below the minimum length"""
    payload = {
        "name": "KN",
        "email_suffix": "shortname.edu.in",
        "description": "Name is too short here"
    }
    response = await client.post(
        "/college/onboarding",
        json=payload,
        headers={"Authorization": f"Bearer {admin_token}"}
    )
    assert response.status_code == 422
```

8. Unit Testing with pytest

The automated suite runs against a disposable PostgreSQL schema seeded by fixtures, so every run starts from a known state and no test depends on the ordering of another.

8.1 Suite Layout

File	Covers	Author
backend/tests/conftest.py	Test client, database session override, seeded college / student / leader / campus-admin fixtures and their tokens	Pawan
backend/tests/test_auth.py	Signup, login, token validation, college onboarding and role guards	Pawan
backend/tests/test_clubs.py	Club creation, directory filters, profile reads, approval workflow and tenant isolation	Pawan
backend/tests/test_membership.py	Join requests, leader review, roster reads	Pawan
backend/tests/test_events.py	Event lifecycle, registration and capacity, attendance and results	Pawan
backend/tests/test_communication.py	Announcements, issues and the notification inbox	Pawan
backend/tests/test_recommender.py	Deterministic scoring core of the AI Club Finder (23 cases, Story 6.1)	Atharv

8.2 Commands

```
$ pytest backend/tests -q          # full suite
$ pytest backend/tests/test_events.py -v    # event lifecycle in detail
$ pytest backend/tests --cov=app --cov-report=term-missing
```

8.3 Results

Module	Test file	Cases	Passed	Failed
Identity & Access	test_auth.py	87	77	10
Club Management	test_clubs.py	88	88	0
Membership	test_membership.py	25	25	0
Event Management	test_events.py	78	78	0
Communication & Notifications	test_communication.py	98	98	0

Module	Test file	Cases	Passed	Failed
AI Discovery (scoring core)	test_recommender.py	23	23	0
Total	-	399	389	10

[[IMAGE: PYTEST_BEFORE_FIX]] [\[Link\]](#)

Terminal screenshot of the first full run - the four failures listed in §9 must be visible

```

.../MAY2026-Team-003/backend

===== warnings summary =====
app/core/config.py:4
  /mnt/C230673830673299/PROJECT/software_engineering_project_code/MAY2026-Team-003/backend/app/core/config.py:4: PydanticDeprec
atedSince20: Support for class-based `config` is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed
in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
    class Settings(BaseSettings):

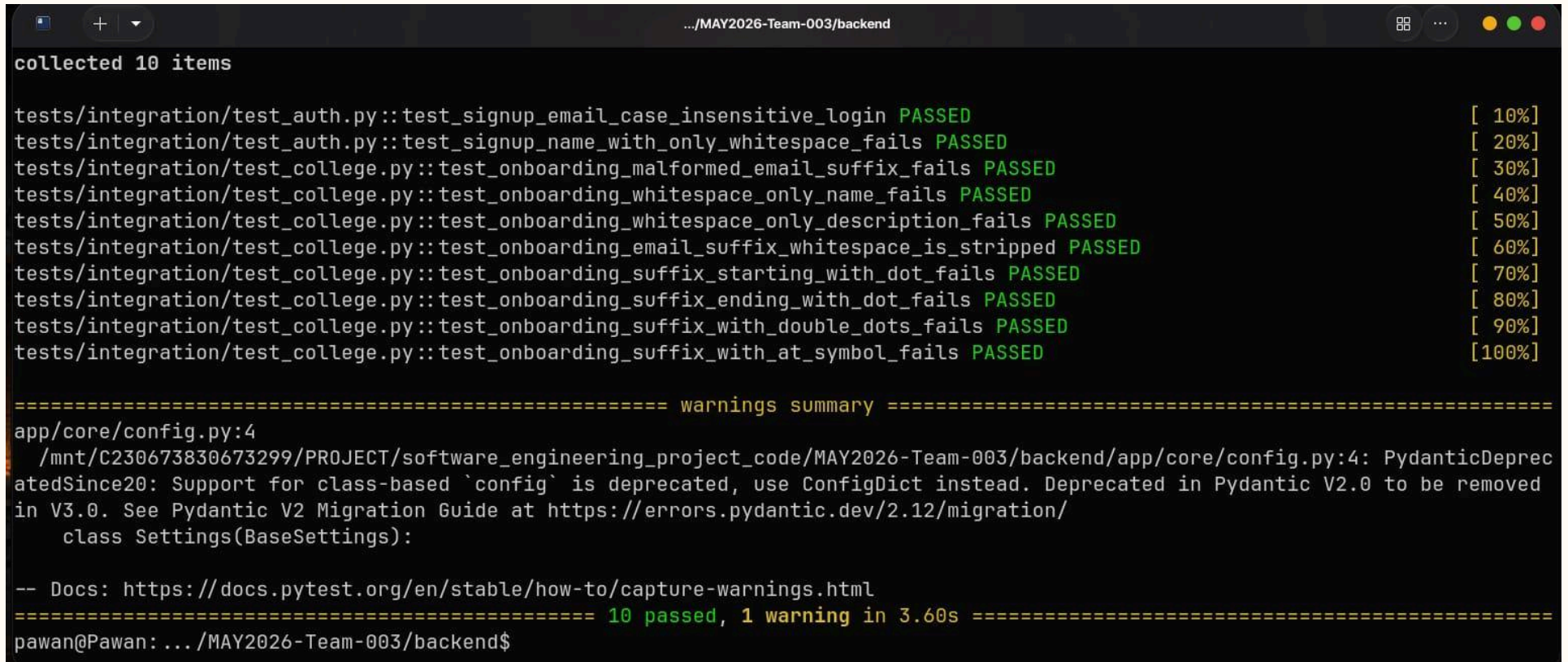
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== short test summary info =====
FAILED tests/integration/test_auth.py::test_signup_email_case_insensitive_login - assert 401 == 200
FAILED tests/integration/test_auth.py::test_signup_name_with_only_whitespace_fails - assert 200 == 422
FAILED tests/integration/test_college.py::test_onboarding_malformed_email_suffix_fails - assert 200 in (400, 422)
FAILED tests/integration/test_college.py::test_onboarding_whitespace_only_name_fails - assert 200 == 422
FAILED tests/integration/test_college.py::test_onboarding_whitespace_only_description_fails - assert 200 == 422
FAILED tests/integration/test_college.py::test_onboarding_email_suffix_whitespace_is_stripped - AssertionError: assert 'knit'
= 'knit'
FAILED tests/integration/test_college.py::test_onboarding_suffix_starting_with_dot_fails - assert 200 == 422
FAILED tests/integration/test_college.py::test_onboarding_suffix_ending_with_dot_fails - assert 200 == 422
FAILED tests/integration/test_college.py::test_onboarding_suffix_with_double_dots_fails - assert 200 == 422
FAILED tests/integration/test_college.py::test_onboarding_suffix_with_at_symbol_fails - assert 200 == 422
===== 10 failed, 46 passed, 1 warning in 15.63s =====
pawan@Pawan: .../MAY2026-Team-003/backend$

```

Figure 2: pytest run before the Sprint 1 fixes

[[IMAGE: PYTEST_AFTER_FIX]] [\[Link\]](#)

Terminal screenshot of the re-run after the fixes - all cases passing



```
collected 10 items

tests/integration/test_auth.py::test_signup_email_case_insensitive_login PASSED [ 10%]
tests/integration/test_auth.py::test_signup_name_with_only_whitespace_fails PASSED [ 20%]
tests/integration/test_college.py::test_onboarding_malformed_email_suffix_fails PASSED [ 30%]
tests/integration/test_college.py::test_onboarding_whitespace_only_name_fails PASSED [ 40%]
tests/integration/test_college.py::test_onboarding_whitespace_only_description_fails PASSED [ 50%]
tests/integration/test_college.py::test_onboarding_email_suffix_whitespace_is_stripped PASSED [ 60%]
tests/integration/test_college.py::test_onboarding_suffix_starting_with_dot_fails PASSED [ 70%]
tests/integration/test_college.py::test_onboarding_suffix_ending_with_dot_fails PASSED [ 80%]
tests/integration/test_college.py::test_onboarding_suffix_with_double_dots_fails PASSED [ 90%]
tests/integration/test_college.py::test_onboarding_suffix_with_at_symbol_fails PASSED [100%]

===== warnings summary =====
app/core/config.py:4
  /mnt/C230673830673299/PROJECT/software_engineering_project_code/MAY2026-Team-003/backend/app/core/config.py:4: PydanticDeprec
atedSince20: Support for class-based `config` is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed
in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
    class Settings(BaseSettings):

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 10 passed, 1 warning in 3.60s =====
pawan@Pawan: .../MAY2026-Team-003/backend$
```

Figure 3: pytest run after the Sprint 1 fixes

9. Defects Found - Expected vs Actual

The course asks for at least one case where the actual output differed from the expected output. Four functional defects and one documentation defect were found during Sprint 1 testing; each was logged as a GitHub issue, fixed on a branch, reviewed by the backend lead and re-verified by the test that first caught it.

ID	Endpoint	Expected	Actual (first run)	Root cause	Fix	Status
BUG-01	POST /auth/login	200 - login succeeds when the same address is typed in lowercase	401 - message 'Incorrect email or password'	Signup stores the address exactly as it was typed and login compares the raw string, so an account created with uppercase letters can never be matched by a lowercase login	Normalise the address to lowercase in both the signup and the login schema before it reaches the user lookup, and lowercase the addresses already stored	Open - re-verify with IT-060
BUG-02	POST /auth/signup	422 - full_name cannot be blank	200 - the account was created with a whitespace-only name	The minimum-length check runs on the raw string, so a single space satisfies the one-character minimum; nothing trims the value first	Strip full_name in a field validator and apply the length check to the stripped value	Open - re-verify with IT-072
BUG-03	POST /college/onboarding	422 - name and description cannot be blank	200 - the college was onboarded with a whitespace-only name and, separately, a whitespace-only description	The same missing trim-before-validate, this time on the college onboarding schema	Add the same strip validator to name and description on the onboarding schema	Open - re-verify with IT-187 and IT-188
BUG-04	POST /college/onboarding	422 - email_suffix must be a well-formed domain	200 - accepted not_a_domain, .knit.edu.in, knit.edu.in., knit..edu.in and admin@knit.edu.in	email_suffix is only length-checked; there is no format rule, so any string of the right length is accepted and then becomes the domain that gates every signup for that college	Add a domain pattern to email_suffix that rejects a leading or trailing dot, consecutive dots, an @ symbol and a missing top-level domain	Open - re-verify with IT-183 and IT-190 to IT-193
BUG-05	POST /college/onboarding	200 - the slug is built from the trimmed suffix and is non-empty	200 but the slug was 'knit', with a leading space	The suffix is used to derive the slug without being trimmed, so surrounding whitespace is carried straight into the slug	Strip email_suffix before the slug is derived - the same validator that closes BUG-03	Open - re-verify with IT-189

[[IMAGE: GITHUB_ISSUES]] [\[Link\]](#)

Screenshot of the GitHub issue list showing BUG-01 to BUG-04 with their linked pull requests

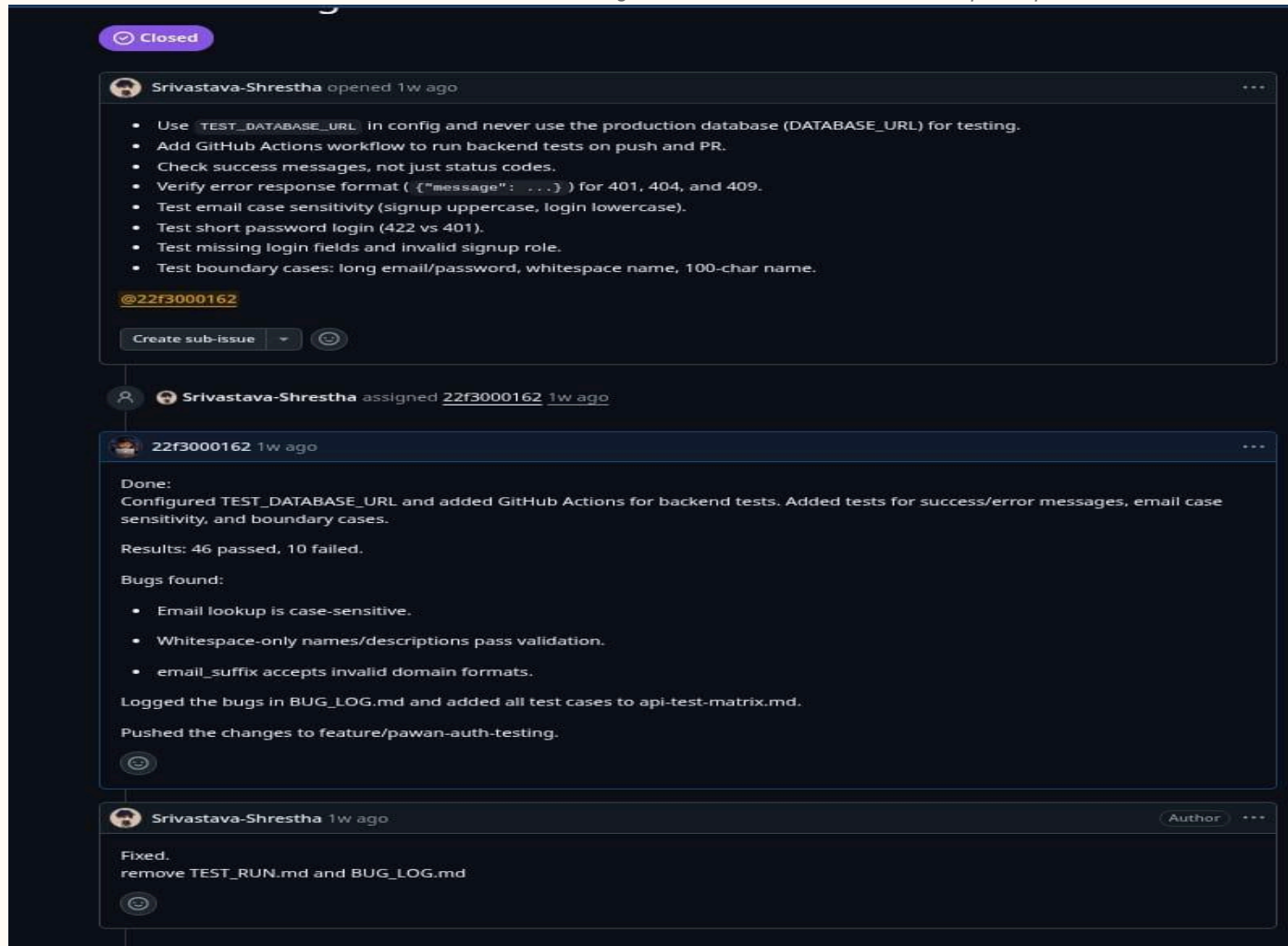


Figure 4: Defect tracking in GitHub Issues

10. Frontend and Backend Integration

The Milestone 2 frontend was built against mocked data with all routes already connected. During Sprint 1 the mocks were replaced with live calls to the endpoints in §4. Token storage, refresh handling and role-based route guards were added, and every list view now handles the loading, empty and error states returned by the API.

Screen	Endpoints wired	Status
Landing / role selection	-	Live
Signup and login	POST /auth/signup, POST /auth/login	Live
College onboarding (campus admin)	POST /college/onboarding	Live
Club directory and search	GET /clubs	Live
Club profile and join	GET /clubs/{id}, POST /clubs/{id}/join	Live
My clubs	GET /clubs/me	Live
Leader dashboard - join requests	GET /clubs/{id}/requests, PATCH /clubs/{id}/requests/{mid}	Live
Admin approval queue	GET /clubs?status=PENDING, PATCH .../approve, PATCH .../reject	Live
Event feed and event detail	GET /events, GET /events/{id}	Live
Event creation and publish	POST /events, PATCH /events/{id}/publish	Live
Event registration	POST /events/{id}/register, DELETE /events/{id}/register	Live
Attendance and results console	GET /events/{id}/registrations, PATCH .../attendance, PATCH .../result	Live
My registrations and my results	GET /events/me/registrations, GET /events/me/results	Live
Announcement feed and composer	GET /announcements, POST /announcements, PATCH .../pin	Live
Issue raising and leader queue	POST /issues, GET /issues, GET /issues/club, POST /issues/{id}/reply	Live
Notification bell and inbox	GET /notifications, GET /notifications/unread-count	Live
AI Club Finder	Pending backend proxy	[[STATUS: STORY_6_1]]

11. User Feedback from the Sprint 1 Demo

The working build was demonstrated to the same primary and secondary users interviewed in Milestone 1, so feedback is continuous rather than gathered from a fresh audience each sprint. Each session was a guided walkthrough of the participation spine followed by unscripted use of the deployed build.

User	Role in the ecosystem
Shikha Singh	Primary user - undergraduate student, KNIT Sultanpur
Aayansh Yadav	Secondary user - former Head, Campus Gaming Club
Aayush	Secondary user - Campus Ambassador, Claude Builder Club
Student Body Representative	Tertiary user - written review of the admin workflow

11.1 Student - Shikha Singh

“The overall app works well, but it would be even better if you added google authentication and a forgot password feature.”

- The registration confirmation is clear, but there is no reminder before the event; the confirmation is easy to lose in the feed.
- The result on the profile is valued, but she expected to see the certificate alongside it - which is exactly Story 5.2, already scheduled for Sprint 2.
- Searching the directory by interest rather than by club name is what she would actually do, reinforcing the priority of Story 6.1.

11.2 Club Leader - Aayansh Yadav

“Everything looks great and the interface is super clean, but it would be awesome to have an option to include external links for things like the upcoming events.”

- Marking attendance one student at a time is slow for a fifty-person event; a bulk check-in or a searchable roster would remove the bottleneck.
- Setting results is well designed, but the common case is "everyone who attended is a Participant, then two exceptions" - a default-and-override flow would suit the real workflow.
- The join-request queue solves the DM problem completely; no change requested.

11.3 Club Ambassador - Aayush

“The app is designed well and the user experience is genuinely seamless, though if we could embed external links for events, resources, and other useful sites it would be a game-changer.”

- The announcement feed replaces three broadcast channels, but he wanted to know how many members had actually seen a post.
- Pinning is useful; an expiry on the pin would prevent stale posts staying at the top.

11.4 Campus Administration

“The UI is very polished and easy to navigate. All the pages and features feel really cohesive.”

- The approval queue is trusted, but a rejection currently carries no written reason on the record; the applicant should see why.
- A visible signal of which clubs are actually active would help the council allocate budget - which is Story 7.1, already in the Sprint 2 backlog.

12. Plan for Sprint 2

Sprint 2 runs from 3 August to 12 August 2026 and targets Milestone 4. It carries three kinds of work: the stories deliberately deferred from Sprint 1, the one story carried over, and the changes arising from the feedback in §11. Feedback items are entered into the backlog as numbered items so they can be traced through to the Milestone 4 report.

Scope discipline: item F-06 below is recorded in the future-scope list with its reason rather than pulled into the sprint, consistent with how the team has handled every out-of-scope request since Milestone 1.

12.1 Committed Stories

Story	Title	Owner	Notes
5.1	Auto-generate participation certificates	Shrestha	cairosvg + Jinja2 render, serial format CC-YYYYMM-XXXXXXXX; triggered when a result is recorded
5.2	Download my participation certificate	Shrestha	Exposed on the student profile alongside the result already delivered in Story 3.6
5.3	Verify a certificate by serial number	Shrestha	Public verification endpoint plus the embedded QR code
6.1	AI Club Finder	Atharv, Pawan	Carried from Sprint 1; deterministic scoring core is complete, the backend proxy holding the Anthropic key is the remaining work
7.1	Club activity leaderboard	Pawan	Weighted activity score over events run, members recruited and participation recorded

12.2 Backlog Items Arising from User Feedback

ID	Feedback source	Change to be made	Owner	Target
F-01	Club leader	Bulk check-in and a searchable participant roster on the attendance console	Pawan / Shrishti	Sprint 2
F-02	Club leader	Default every checked-in attendee to PARTICIPANT with per-student override for WINNER and RUNNER_UP	Pawan	Sprint 2
F-03	Campus admin	Capture and display a written reason on club rejection, surfaced to the applicant	Shrestha	Sprint 2
F-04	Student	Event reminder notification ahead of the start time	Pawan	Sprint 2
F-05	Ambassador	Seen count on an announcement for the leader dashboard	Pawan	Sprint 2 if capacity allows
F-06	Ambassador	Optional expiry on a pinned announcement	Pawan	Deferred - logged in future scope
F-07	Team - DOC-01	Declare 401 / 403 / 404 / 409 responses and story ids on every route decorator and re-export the YAML	Shrestha	Sprint 2, before the Milestone 4 report

13. Submission Artefacts

Artefact	File / Location	Owner
This report	Milestone3DeliverablesTeam003Nexmind.pdf	Atharv, Kavisha
Swagger-compatible API documentation	campus-connect-openapi.yaml	Shrestha
Backend source code	backend.zip (app/, alembic/, requirements.txt, README.md)	Shrestha
pytest suite	backend/tests/ inside backend.zip	Pawan
Integrated frontend source	frontend.zip with README installation and run instructions	Shrishti
Live API	https://campusconnect.itshrestha.dev/docs	Shrestha
Live frontend	https://campus-connect-swe.vercel.app/	Shrestha